# Manual Secure Code Review

## 1. Authentication

1.1. Weak Password Policy
1.2. No Brute Force Protection
1.3. No Captcha Available
1.4. Lack of Rate Limiting
1.5. Weak Password Hashing
1.6. Session Fixation
1.7. Session Token in URL
1.8. No Validation of OAuth/OpenID (OIDC)
1.9. Password Reset Token Not Expiry
1.10. Missing MFA
1.11. Bypass 2FA
1.12. Hardcoded Credentials in Source Code
1.13. No Account Logout Logic
1.14. Password Reset Token Without Verification
1.15. No CSRF Token in Sensitive Action
1.16. Using Weak JWT Secret
1.17. Missing Token Expiration & Weak Algorithm for JWT
1.18. Long-Lived Tokens Without Rotation

## 2. Authorization

2.1. IDOR (Insecure Direct Object Reference)
2.2. BOLA (Broken Object-Level Authorization)
2.3. Missing Authorization on Endpoint
2.4. Forceful Browsing
2.5. Improper RBAC (Role-Based Access Control)
2.6. BFLA (Broken Function-Level Authorization)
2.7. Weak Access Permissions
2.8. Mass Assignment
2.9. User Enumeration

## 3. Cryptographic Failures

3.1. Weak Hashing Algorithms (MD5, SHA1)
3.2. No Salting for password hashing
3.3. Hardcoded credentials & secrets leakage
3.4. Insecure Key Management
3.5. Insecure Randomness
3.6. Improper password storage
3.7. Improper use of encryption modes
3.8. Insecure protocol usage
3.9. Lack of digital signatures or integrity protection
3.10. Replay and session token issues
3.11. Cryptographic padding issues
3.12. Sensitive data exposure in logs (secrets leakage
3.13. Insufficient entropy
3.14. Weak cipher suite enforced
3.15. Use of base64 as encryption
3.16. JWT with 'none' algorithms accepted
3.17. JWT signed with weak secret (short key)
3.18. JWT without expiration
3.19. Improper signature verification
3.20. JWT KID Injection
3.21. No Secure flag in cookie
3.22. No HttpOnly flag in cookie
3.23 Missing SameSite attribute in cookie

## 4. Input Validation & Sanitization(Injection)

4.1. Lack of Input Validation & Sanitization
4.2. Weak Input Validation & Sanitization
4.3. Cross-Site Scripting (XSS)
4.4. SQL Injection
4.5. LFI, RFI & Directory Traversal
4.6. NoSQL Injection
4.7. XXE / XML Injection
4.8. Server-Side Request Forgery (SSRF)
4.9. Command Injection
4.10. Server-Side Template Injection (SSTI)
4.11. LDAP Injection
4.12. CRLF Injection
4.13. Host Header Injection
4.14. Null Byte Injection (via Host Header, LFI)
4.15. HTTP Request Smuggling
4.16. Prototype Pollution
4.17. Unrestricted File Upload
4.18. Deserialization (Insecure)
4.19. Regular Expression

# Manual Secure Code Review Approach and Methodology

- Before starting the security code review, I discuss with the development team to understand the technology stack (such as Java, .NET, PHP, or Python), core functionalities (like Registration and Login, Search, File Upload, Dashboard, Account Setting), architecture, and business objectives. This discussion helps to identify potential risk areas and understand how authentication, authorization, and session management are designed and implemented.

- I collect and analyze the source code, configuration files (e.g., web.config, application.properties, .env), third-party dependency files (pom.xml, package.json), and environment details to review how the application handles data flow, authentication, and secure communication (e.g., HTTPS enforcement and secure cookie flags).

- I focus on authentication mechanisms, assessing how user identities are verified and managed, including login workflows, multi-factor authentication, password storage and validation, token generation, and session handling. I verify the use of secure password hashing algorithms such as bcrypt, PBKDF2, or Argon2, and check for account lockout, brute-force protection, and secure token handling to prevent replay, fixation, or session hijacking attacks.

- Next, I analyze the application flow, identifying entry points such as user inputs, APIs, forms, and file uploads, and trace how requests pass through controllers, business logic, and data layers. This helps map trust boundaries and detect critical attack surfaces where untrusted data interacts with sensitive components.

- During the manual review, I trace data flow from source to sink, focusing on validation, sanitization, and data storage practices. I look for unvalidated inputs, direct query concatenations, insecure function calls, and sensitive data exposure—identifying cases where untrusted input may reach sensitive functions or external interfaces without proper protection.

- I perform keyword-based searches for risky functions like executeQuery, Runtime.exec, eval, sendRedirect, and FileInputStream to verify secure coding practices such as prepared statements, parameterized queries, output encoding, and input sanitization.

- I also review configuration and deployment settings for weaknesses such as insecure headers, misconfigured CORS, or missing TLS enforcement. Authentication and session management reviews include checking session expiration, logout mechanisms, cookie attributes (Secure, HttpOnly, SameSite), and CSRF protection.

- Finally, I align findings with OWASP Top 10 and CWE categories, ensuring authorization and business logic controls are enforced. Each vulnerability is documented with file path, line reference, impact, severity (CVSS), and remediation recommendations, followed by a detailed discussion with developers to validate logic and finalize secure fixes.

# 1. Authentication

## 1.1. Weak Password Policy

**Impact:** Account takeover from easy-to-guess passwords; weak defense against credential stuffing.

**Where to check**

- Registration handlers and password-change endpoints (`RegisterController`, `SignupController`, `UserService`, `authController`, `views.py`, `routes/signup.js`)
- Shared validation utilities (`password_validator.*`, `validators/`, `utils/security/`)
- Frontend forms and client-side JS (NOTE: server-side must be authoritative)
- CI pipeline policy checks or policy-as-code

**What to check**

- Is password policy enforced server-side (not only JS)?
- Policy specifics: min length (≥12 recommended) OR passphrase rules, complexity, banned/common password list, minimum entropy.
- Any call to breach-check services (e.g., haveibeenpwned) or local deny-list.

**Vulnerable function/method patterns**

- `if (password.length() > 3)`
- `allowSimplePassword()`
- `register(user)` with no call to `validatePassword(...)`

**Secure function/method patterns**

- `validatePasswordPolicy(password)`
- `checkPasswordStrength(password)`
- `isBreachedPassword(password)` (calls external/local breach list)

### Vulnerable Code Example

```
// vulnerable – only checks length 6
if (password.length >= 6) createUser(username, password);
```

### Secure Code Example

```
// secure (Java-style)
if (!PasswordPolicy.validate(password)) {
  throw new InvalidPasswordException("Password fails policy");
}
// PasswordPolicy enforces length >=12 or passphrase rules and checks deny-list
```

## 1.2. No Brute Force Protection

**Impact:** Attackers can perform unlimited guesses; easy compromise through credential stuffing.

**Where to check**

- Login endpoints, auth services (`login()`, `authenticate()`, `AuthController`, `AuthService`)
- Cache/Redis code for counters, DB columns `failed_login_attempts`, `locked_until`
- API Gateway / edge rate-limiters (NGINX, CloudFront, API Gateway)

**What to check**

- Are failed attempts recorded per user and per IP?
- Is there account lockout or progressively increasing delay?
- Is error messaging consistent (no user enumeration)?

- Is there per-IP throttling at edge level?

**Vulnerable function/method patterns**

- `if (checkPassword(...)) loginUser();` with no failure handling
- No calls to `incrementFailedAttempt()` or `isBlocked()` in failure branch

**Secure function/method patterns**

- `incrementFailedAttempt(username, ip)`
- `isAccountLocked(username)`
- `lockAccount(username, duration)`
- `throttleByIP(ip)` or `rateLimit(key)`

### Vulnerable Code Example

```
# vulnerable: no failure tracking
def login(username, password):
    if check_password(username, password):
        session.create(username)
```

### Secure Code Example

```
// secure: increment and lock logic (Node-style)
if (loginAttemptService.isBlocked(username, ip)) return errorTooManyRequests();
if (!authService.verifyPassword(username, password)) {
  loginAttemptService.incrementFailedAttempt(username, ip);
  return errorUnauthorized();
}
loginAttemptService.resetFailedAttempts(username);
createSession(user);
```

**Code-level remediation**

1. Add per-account failed-attempt counters stored in DB/Redis.
2. After N failures (e.g., 5), lock account for exponential backoff (15m, 1h).
3. Implement per-IP rate-limiting at the gateway and per-account throttling in-app.
4. Use consistent error responses to avoid username enumeration.
5. Add admin unlock and automated unlock flows.

**Detection & review tips**

- Look for missing counter increments in login failure path.
- Trace `authenticate()` call chain and check both success and failure branches.

## 1.3. No Captcha Available

**Impact:** Bots can automate account abuse (credential stuffing, mass registration, reset abuse).

**Where to check**

- Login, register, forgot-password server handlers and their middleware
- Any adaptive risk module that should trigger CAPTCHA

**What to check**

- Is server-side CAPTCHA verification present and enforced (not just UI)?
- Is CAPTCHA triggered adaptively (after threshold or when risk signals appear)?

**Vulnerable function/method patterns**

- **No `verifyCaptcha()`** or server calls to Google/hCaptcha in auth endpoints.

**Secure function/method patterns**

- `isCaptchaRequired(user/ip)` and `verifyCaptcha(token)`

## Vulnerable Code Example

```
// vulnerable: client shows CAPTCHA, but server never verifies
app.post('/login', (req,res) => { if (checkPassword(...)) res.send('ok'); });
```

## Secure Code Example

```
if (captchaService.isCaptchaRequired(username, ip)) {
  captchaService.verify(token); // server-side call to provider
}
```

**Code-level remediation**

1. Implement server-side verification for CAPTCHA tokens from provider.
2. Trigger CAPTCHA after failed attempts or when heuristics indicate bot behavior.
3. Keep UX in mind (invisible captcha / progressive challenge).

**Detection & review tips**

- Find form POST handlers and inspect whether they validate a `captchaToken` parameter server-side.

# 1.4. Lack of Rate Limiting

**Impact:** Brute-force, resource exhaustion, API abuse and DoS.

**Where to check**

- API gateway / edge (NGINX, Cloudflare, API GW) configs
- App middleware for throttling (Express middleware, Spring filters)
- Per-route settings for auth endpoints

**What to check**

- Is there global + route-specific rate limiting (especially `/login`, `/forgot-password`)?
- Are `Retry-After` or `X-RateLimit-*` headers returned?

**Vulnerable function/method patterns**

- No middleware or filters named `rateLimit`, `throttle`, `limiter`

**Secure function/method patterns**

- `rateLimitMiddleware({windowMs, max})`, token-bucket or leaky-bucket implementations

## Vulnerable Code example

```
app.post('/login', loginHandler); // no rate-limiter applied
```

## Secure Code example

```
app.post('/login', rateLimit({windowMs:60000, max:5}), loginHandler);
```

**Code-level remediation**

1. Add gateway-level (edge) rate-limits and app-level per-route throttling.
2. Apply stricter limits to auth and password-reset endpoints.
3. **Return 429** and set `Retry-After`.
4. Use IP + account keys to avoid blocking shared proxies.

**Detection & review tips**

- Inspect routing and middleware stacks for presence of rate limiting on auth routes.

**Logging/monitoring**

- Record 429 counts, unusual traffic bursts, and source IPs.

## 1.5. Weak Password Hashing

**Impact:** Compromised DB leads to fast cracking and mass account takeover.

**Where to check**

- Password hashing utilities, registration / change flows, DB stored hash formats (users table), migration scripts

**What to check**

- Are passwords hashed with fast algorithms (MD5/SHA1/SHA256) or proper KDFs?
- Is per-user salt used? Are parameters (cost) appropriate?

**Vulnerable function/method patterns**

- `md5(password)`, `sha1(password)` or `crypto.createHash('sha256')` with no salt

**Secure function/method patterns**

- `bcrypt.hash(password, cost)`, `argon2.hash(password, options)`, `pbkdf2(password, salt, iterations)`, `verifyPassword(plain, storedHash)`

### Vulnerable Code example

```
MessageDigest md = MessageDigest.getInstance("MD5");
byte[] digest = md.digest(password.getBytes());
```

### Secure Code example

```
from argon2 import PasswordHasher
ph = PasswordHasher(time_cost=3, memory_cost=65536, parallelism=4)
hash = ph.hash(password)
```

**Code-level remediation**

1. Adopt Argon2id (preferred) or bcrypt/PBKDF2 with strong parameters.
2. Store algorithm metadata in user record for migration.
3. Implement on-login rehash: when a user logs in with old hash, re-hash with new algorithm and save.
4. Use server-side pepper if necessary (stored in vault), and rotate with plan.

**Detection & review tips**

- Inspect DB hashes: bcrypt starts `$2a$/$2b$`, argon2 starts `$argon2`, PBKDF2 formats vary.

**Logging/monitoring**

- Alert if code uses deprecated hashing APIs.

## 1.6. Session Fixation

**Impact:** Attacker can reuse a session fixed before login to impersonate user after they authenticate.

**Where to check**

- Session handling post-login, session create/invalidate code (**`SessionManager`**, **servlet filters**), SSO callback logic

**What to check**

- Is session id regenerated after authentication/privilege escalation?
- Are **`HttpOnly`**, **`Secure`**, and **`SameSite`** cookie flags set?

**Vulnerable function/method patterns**

- **`session.setAttribute("user", user)`** without **`session.invalidate()`** or **`regenerateSessionId()`**

**Secure function/method patterns**

- **`invalidateSession(oldId)`** + **`createNewSession()`** **or** **`regenerateSessionId()`**; cookie flags set

**Vulnerable Code example**

```
request.getSession().setAttribute("userId", user.getId()); // reuses current session
```

## Secure Code Example

```
HttpSession old = request.getSession(false);
if (old != null) old.invalidate();
HttpSession s = request.getSession(true);
s.setAttribute("userId", user.getId());
```

**Code-level remediation**

1. Invalidate any existing session and create a fresh session immediately upon successful authentication.
2. Set cookie attributes: **`HttpOnly`**, **`Secure`**, **`SameSite=Strict/Lax`** depending on app flow.
3. If using JWT, ensure creation of new token after login and discard any pre-auth token.

**Detection & review tips**

- Capture cookie/session id before and after successful login; confirm change.

## 1.7. Session Token in URL

**Impact:** Token leakage via referer headers, logs, or browser history — leads to session theft.

**Where to check**

- Redirects, link builders, return URL code, SSO flows, email links

**What to check**

- Are tokens appended to query strings (`?token=` or `?session=`)?
- Are redirect handlers passing token in URL or fragment?

**Vulnerable function/method patterns**

- **`res.redirect('/app?token=' + token)`**
- **`window.location = '/app#token=' + token`** (fragment is slightly safer but still risky)

**Secure function/method patterns**

- **`setCookie('session', token, { httpOnly: true, secure:true })`** or send tokens in Authorization header for API

### Vulnerable Code Example:

```
https://example.com/app?sessionid=abc123
```

### Secure Code Example:

```
res.cookie('session', token, { httpOnly: true, secure: true, sameSite: 'Lax' });
res.redirect('/app');
```

#### Code-level remediation

1. Never include auth tokens in URL query strings.
2. Use HttpOnly Secure cookies or Authorization header for token transport.
3. If using short one-time codes in URL (e.g., magic links), exchange them immediately for cookie or token server-side and remove query param (redirect to clean URL).

#### Detection & review tips

- Inspect redirects and email templates for **`token=`** or **`session=`**

## 1.8. No Validation of OAuth/OpenID (OIDC)

**Impact:** Forged tokens or replayed assertions can lead to account takeover via SSO.

#### Where to check

- OIDC/OAuth callback handlers, token verification utilities, JWKS key fetching code, `state`/`nonce` storage and verification

#### What to check

- Are id_tokens verified for signature using provider JWKS?
- Are claims validated: **`iss`**, **`aud`** (client id), **`exp`**, **`iat`**, and **`nonce`**?
- Is **`state`** checked to prevent CSRF in callback?

#### Vulnerable function/method patterns

- **`jwt.decode(id_token)`** used directly to accept claims without signature verification
- No **`verifySignature()`** call

#### Secure function/method patterns

- **`verifyIdToken(id_token, issuer, clientId)`** that checks signature & claims
- **`validateOAuthState(storedState, receivedState)`**

### Vulnerable Code example

```
// vulnerable: naive decode
const claims = jwt.decode(id_token);
if (claims.email) loginUser(claims.email);
```

### Secure Code Example

```
const verified = await oidcClient.verifyIdToken({ idToken: id_token, audience:
CLIENT_ID });
if (!verified) throw new Error('Invalid token');
```

#### Code-level remediation

1. Always verify the JWT signature (use library that fetches JWKS).
2. Check **`iss`**, **`aud`**, **`exp`**, **`nonce`** & **`state`**.
3. Map external identities to local accounts with safeguards (do not auto-assign admin privileges).
4. Log SSO events and rejections.

**Detection & review tips**

- Look for callback handlers that skip **`verify`** or rely only on decoded payload.

## 1.9. Password Reset Token Not Expiry

**Impact:** Lifetime or predictable reset tokens allow account takeover.

**Where to check**

- **`forgot-password`** and **`reset-password`** handlers, password_reset DB/table, email templates

**What to check**

- Are tokens cryptographically random **(≥128 bits**) and stored hashed?
- Do tokens have expiry times and one-time-use semantics (**`used`** flag)?
- Are tokens invalidated after use and after password change?

**Vulnerable function/method patterns**

- **`resetToken = user.email + timestamp`** or **`UUID()`** used with no expiry
- Token stored plaintext in DB

**Secure function/method patterns**

- **`token = randomBytes(32)`**, **`db.save(hash(token), userId, expires_at`**)
- **`validateResetToken(hash(token))`** then **`invalidateResetToken()`**

### Vulnerable Code example

```
token = user.email + str(int(time.time()))
db.save(user.id, token)  # no expiry
```

### Secure Code Example

```
token = secrets.token_urlsafe(32)
token_hash = hashlib.sha256(token.encode()).hexdigest()
db.insert('password_resets', user_id=user.id, token_hash=token_hash,
expires_at=now+1h, used=False)
# send 'token' to user's email (not logged)
```

**Code-level remediation**

1. Generate cryptographically secure random tokens and send token only via email.
2. Hash tokens before storing in DB and set short expiry (1 hour standard).
3. Mark token used on successful reset and reject reuse.
4. Remove tokens from logs and URLs (redirect to tokenless pages after exchange).

**Detection & review tips**

- Inspect DB **`password_resets`** table for plaintext tokens or no expiry column.

**Logging/monitoring**

- Log reset request counts per account and per IP, alert when unusual volume.

## 1.10. Missing MFA

**Impact:** Account compromise when primary credential is stolen.

**Where to check**

- Login flow, MFA enrollment endpoints, **`mfa`** service, user model (**`mfa_enabled`**, **`mfa_secret`**) and backup codes management

**What to check**

- Is there server-side enforcement of MFA before granting full session?
- Are **TOTP** / WebAuthn implemented securely? Are backup codes hashed and single-use?

**Vulnerable function/method patterns**

- **`if (checkPassword(username, password)) { createSession(username) }`** (no MFA step)

**Secure function/method patterns**

- **`startMfaChallenge(user)`**, **`verifyTotpCode(user, code)`**, **`requireMfaFor(userId)`**

## Vulnerable Code Example

```
// vulnerable: single-factor only
if (auth.verifyPassword(user, pass)) {
  createSession(user);
}
```

## Secure Code Example

```
// secure: two-phase
if (auth.verifyPassword(user, pass)) {
  session.setAttribute("authPhase", "mfa_required");
  // present TOTP challenge
}
if (verifyTotp(user, code)) {
  session.setAttribute("authPhase", "fully_authenticated");
}
```

**Code-level remediation**

1. Add optional and/or mandatory MFA flows: TOTP (RFC6238), U2F/WebAuthn for highest assurance.
2. Ensure server verifies OTP and stores secrets encrypted; backup codes hashed and single-use.
3. Enforce MFA for critical operations (changing auth, high-risk actions).

**Detection & review tips**

- Search login path for any reference to `mfa`, **`otp`**, **`totp`**, **`webauthn`**. If absent, probably not implemented.

## 1.11. Bypass 2FA

**Impact:** Attackers can skip second factor leading to complete compromise.

**Where to check**

- Session state flags, auth flow state machine, protected route guards, backup code consumption

**What to check**

- Are protected routes checking **`mfa_verified`** or only **`authenticated`**?
- Is **`authenticated`** set before `mfa` verified?
- Are backup codes enforced single-use & properly validated?

**Vulnerable function/method patterns**

- **`session.setAttribute("authenticated", true)`** before **`verifyMFA()`**
- Protected routes check only **`if (session.authenticated) ...`**

**Secure function/method patterns**

- `if (!session.mfaVerified) reject()` in guards; `consumeBackupCode(user, code)` marks used

**Vulnerable Code Example**

```
session.setAttribute("authenticated", true); // done immediately after password check
// later: mfa check optional
```

## Secure Code Example

```
// partial session
session.phase = 'password_ok';
// after OTP
session.phase = 'fully_authenticated';
// route guard checks session.phase === 'fully_authenticated'
```

**Code-level remediation**

1. Implement explicit auth phases and ensure every protected resource checks for the final `mfaVerified` / `fully_authenticated` state.
2. Ensure backup codes are hashed and marked as consumed on use.
3. Review all “fast paths” and API endpoints that may accept alternate auth headers.

**Detection & review tips**

- Search for `authenticated` or boolean auth flags and audit where they’re set and checked.

**Logging/monitoring**

- Log attempts to access protected endpoints when `mfaVerified` false.

## 1.12. Hardcoded Credentials in Source Code

**Impact:** Credential leak leads to immediate compromise of services, DBs, cloud accounts.

**Where to check**

- Source files, config files, `.env`, CI/CD YAML, Dockerfiles, Kubernetes manifests, test fixture files, commit history

**What to check**

- Any plaintext secrets in repository or build artifacts.
- Use of environment variables vs hardcoded literals.

**Vulnerable function/method patterns**

- `const DB_PASS = "topsecret";` or `aws_secret = "..."` in code

**Secure function/method patterns**

- `process.env.DB_PASS`, `secretsManager.getSecret("db/credentials")`

**Vulnerable Code example**

```
String dbPass = "Admin123!";
```

## Secure Code Example

```
const dbPass = process.env.DB_PASSWORD; // secret injected at runtime or retrieved
from vault
```

**Code-level remediation**

1. Remove secrets from repo, rotate them immediately.

2. Move secrets to a vault (AWS Secrets Manager, HashiCorp Vault) and fetch at runtime or inject via secure env.
3. Add secret scanning to CI, and add pre-commit hooks to block secrets.

**Detection & review tips**

- Grep for obvious keywords: **`password=`**, **`DB_PASS`**, **`API_KEY`**, **`SECRET`**, **`PRIVATE_KEY`**.
- Run automated secret scanners.

## 1.13. No Account Logout Logic

**Impact:** Session reuse post-logout allows account persistence and theft.

**Where to check**

- Logout endpoint, session store, refresh token table, token revocation code, SSO logout handlers

**What to check**

- Does logout call **`invalidateSession()`** and revoke refresh token(s)?
- For JWTs: is there a revocation strategy or short TTL + rotation?

**Vulnerable function/method patterns**

- Only **`clearCookie('session')`** client-side without server session invalidation

**Secure function/method patterns**

- **`sessionStore.destroy(sessionId)`** and **`revokeRefreshToken(tokenId)`**

### Vulnerable Code Example

```
res.clearCookie('session');
res.send('logged out');
```

### Secure Code Example

```
sessionStore.invalidate(sessionId);
refreshTokenService.revokeAll(userId);
res.clearCookie('session');
```

**Code-level remediation**

1. Invalidate server-side sessions and revoke refresh tokens on logout.
2. For stateless tokens, keep short access token lifetimes and use revocation lists for refresh tokens or rotate tokens on use.

**Detection & review tips**

- Test: get token, logout, then attempt reuse — if accepted, logout is ineffective.

## 1.14. Password Reset Token Without Verification

**Impact:** Allows direct change of password without verifying user ownership of email/account.

**Where to check**

- Reset endpoint, token verification function, mail sending logic

**What to check**

- Does reset operation require a valid token bound to the user before changing password?
- Does token have expiry and single-use enforcement?

**Vulnerable function/method patterns**

- `resetPassword(username, newPassword)` without `verifyResetToken(token, username)`

**Secure function/method patterns**

- `validateResetToken(token)` returns userId, then `updatePassword(userId, newPass)`

## Vulnerable Code Example

```
if (isset($_POST['username']) && isset($_POST['newPass'])) {
  updatePassword($_POST['username'], $_POST['newPass']); // no token check
}
```

## Secure Code Example

```
user_id = validate_reset_token(token)
if user_id:
    change_password(user_id, new_password)
    mark_token_used(token)
```

**Code-level remediation**

1. Reset flow must be: request -> emailed token -> user clicks link -> server validates token and user before allowing new password -> invalidate token after use.
2. Disallow password change without token or without re-authentication for highly privileged accounts.

**Detection & review tips**

- Inspect reset endpoint signature: if it lacks token param or doesn't check token DB, flag.

## 1.15. No CSRF Token in Sensitive Action

**Impact:** Attackers can coerce authenticated users to perform unwanted actions.

**Where to check**

- Any cookie-authenticated POST/PUT/DELETE endpoints that change state (password, email, funds transfer), HTML forms, SPA CORS settings

**What to check**

- Are CSRF tokens generated server-side and validated on POST?
- Is cookie `SameSite` set appropriately? For APIs, are tokens expected in Authorization header rather than cookies?

**Vulnerable function/method patterns**

- No `csrfMiddleware` usage or no `verifyCsrfToken(req)` call on state-changing endpoints

**Secure function/method patterns**

- `csrfProtectionMiddleware()` in app/global middleware; `validateCsrfToken(token, session)`

## Vulnerable Code Example

```
# (server has no CSRF validation)

<form action="/change-password" method="POST">
  <input name="newPassword">
  <button type="submit">Change</button>
```

```
</form>
```

### Secure Code Example

```
// Express + csurf
app.use(csurf());
app.post('/change-password', csrfProtection, changePasswordHandler);
```

**Code-level remediation**

1. Add CSRF protection middleware for cookie-authenticated routes.
2. For SPA APIs, prefer Authorization header bearer tokens and configure CORS properly.
3. Set cookie **`SameSite=Lax/Strict`** and **`HttpOnly`**.

**Detection & review tips**

- Inspect app startup for CSRF middleware and check forms for **`csrfToken`** fields.

## 1.16. Using Weak JWT Secret

**Impact:** Attackers can forge tokens and escalate privileges.

**Where to check**

- JWT signing configuration: **`.env`** or **`application.yml`** variables like **`JWT_SECRET`** and token creation code (JwtUtil, AuthService)

**What to check**

- Is the secret sufficiently long and random (**≥256** bits for HMAC)?
- Are secrets stored in config files in plaintext or code?

**Vulnerable function/method patterns**

- **`jwt.sign(payload, 'secret')`** or **`JWT_SECRET = 'changeme'`**

**Secure function/method patterns**

- Use RS256/ES256 with private key in vault OR HMAC with a randomly generated 256+ bit secret from vault: **`jwt.sign(payload, privateKey, { algorithm: 'RS256' })`**

### Vulnerable Code Example

```
jwt.sign(payload, '12345'); // insecure short secret
```

### Secure Code Example

```
String token = Jwts.builder().setSubject(user).signWith(privateKey,
SignatureAlgorithm.RS256).compact();
```

**Code-level remediation**

1. Replace weak shared secrets with either: asymmetric keypairs (RS256) or a very high-entropy HMAC secret (>256 bits) stored in a vault.
2. Add key identifiers (**`kid`**) and rotation process.
3. Ensure secrets are not in source or unencrypted config.

**Detection & review tips**

- Inspect env/config and token sign calls to confirm secret source and length.

## 1.17. Missing Token Expiration & Weak Algorithm for JWT

**Impact:** Long-lived tokens enable prolonged unauthorized access; weak algorithms enable token forgery.

**Where to check**

- Token issuance code, verification code, and config for expiration values

**What to check**

- Does token include `exp` and is verification enforcing expiry?
- Is algorithm enforced during verification (do not accept token-supplied `alg` blindly)?
- Are JWTs short-lived (access token ~15m) with refresh tokens for session continuation?

**Vulnerable function/method patterns**

- **`jwt.sign(payload)`** with no **`expiresIn`** or verifying with **`jwt.verify(token`**) without enzyme to enforce algorithms

**Secure function/method patterns**

- **`jwt.sign(payload, key, { expiresIn: '15m', algorithm: 'RS256' })`**
- **`jwt.verify(token, publicKey, { algorithms: ['RS256'] })`**

### Vulnerable Code Example

```
const token = jwt.sign({ sub: id }, secret); // no expiry
```

### Secure Code Example

```
const token = jwt.sign({ sub: id }, privateKey, { algorithm: 'RS256', expiresIn:
'15m' });
jwt.verify(token, publicKey, { algorithms: ['RS256'] });
```

**Code-level remediation**

1. Set reasonable `exp` values: access tokens short (10–30 minutes); refresh tokens longer (days), with rotation.
2. Enforce accepted algorithms (**`alg`**) during verification. Reject `none`.
3. Include **`iat`**, **`nbf`** claims as appropriate.

**Detection & review tips**

- Inspect token generation calls and verification code for missing expiry or algorithm checks.

## 1.18. Long-Lived Tokens Without Rotation

**Impact:** Stolen refresh tokens give long-term access if they are not rotated/invalidated.

**Where to check**

- Refresh token store (DB), token refresh endpoints, device/session binding code

**What to check**

- On refresh, is old refresh token invalidated and a new one emitted (rotation)?
- Are refresh tokens bound to **`jti`**/device id and stored server-side for revocation?

**Vulnerable function/method patterns**

- Issuing static long-lived **`refreshToke`**`n` that never changes or lacks server-side state

**Secure function/method patterns**

- **`rotateRefreshToken(oldJti)`** + **`invalidateRefreshToken(oldJti)`** + store `jti` with device and expiry

## Vulnerable Code Example

```
// vulnerable: static refresh token re-used
res.json({ accessToken, refreshToken: user.refreshToken });
```

## Secure Code Example

```
// secure rotate
if (!refreshStore.validate(oldToken)) deny();
refreshStore.invalidate(oldToken);
const newToken = issueRefreshToken(user);
refreshStore.store(newToken);
```

**Code-level remediation**

1. Implement refresh-token rotation on use and mark previous token as revoked.
2. Bind refresh tokens to device/session (IP/device fingerprint) where feasible.
3. Detect reuse (old token used after rotation) and revoke all tokens for the user on suspected compromise.

**Detection & review tips**

- Inspect refresh endpoint; check whether it invalidates old tokens.

# 2. Authorization (Broken Access Control)

## 2.1. IDOR (Insecure Direct Object Reference)

**Where to Check**

- API endpoints that take IDs in the URL (**`/orders/{id}`**, **`/users/{userId`**}).
- Repository/DAO methods that fetch objects by ID (`findById`, `findOne`, `select * from`).

**What to Check**

- Does the function **validate ownership** of the resource (e.g., **`order.ownerId == currentUser.id`**)?
- Is there **role validation** (e.g., admin can bypass ownership)?
- Is sensitive data returned **before** authorization checks are done?

**Vulnerable Functions**

- **`findById()`** or **`findOne()`** used without verifying the object belongs to the logged-in user.
- Direct SQL queries like **`SELECT * FROM orders WHERE id=?`** executed without validating the requesting user.

**Secure Functions**

- Functions wrapped with ownership checks before returning objects.
- Middleware like **`checkOwnership`**, **`ensureRole`**, or service-layer validation enforcing object-level rules.

**Vulnerable Indication**

- Object is returned immediately after a DB call.

- No reference to **`req.user.id`**, **`principal.getName()`**, or session information in the authorization logic.

**Secure Indication**

- Explicit ownership validation (**`if object.ownerId == user.id`**).
- Role-based override for privileged users (**`if user.role == "admin"`**).
- Errors are returned (`403 Forbidden`) when unauthorized.

## Vulnerable Code Example

```
// Vulnerable: Anyone can access any order by changing the ID
app.get('/orders/:id', async (req, res) => {
  // Fetching order directly using user-supplied ID (vulnerable function)
  const order = await db.orders.findById(req.params.id);

  // Indication of vulnerability: No ownership or role check
  res.json(order);
```

## Secure Code Example

```
// Secure: Validate ownership before returning order
app.get('/orders/:id', async (req, res) => {
  const order = await db.orders.findById(req.params.id);
  if (!order) return res.status(404).send("Not Found");

  // Secure indication: ownership check + admin override
  if (order.ownerId !== req.user.id && !req.user.roles.includes("admin")) {
    return res.status(403).send("Forbidden");
  }

  // Only authorized users get the response
  res.json(order);
});
```

## 2.2. BOLA (Broken Object-Level Authorization)

**Where to Check**

- API endpoints that modify or delete data (**`PUT /comments/{id}`**, **`DELETE /posts/{id}`**).
- Service/repository layer methods: **`deleteById`**, **`updateById`**, **`findByPk.`**

**What to Check**

- Is the logged-in user validated before modifying or deleting the resource?
- Are role-based rules enforced (e.g., moderators can delete any comment)?
- Does the system prevent one user from deleting another's resource?

**Vulnerable Functions**

- Direct calls to **`deleteById()`**, **`updateById()`** without ownership check.
- Service functions that modify resources using only client-provided IDs.

**Secure Functions**

- Wrapper methods like **`deleteCommentIfOwner()`** or **`updatePostIfAuthorized`**().
- Security annotations like **`@PreAuthorize("hasRole('ADMIN')")`**.

- Middleware checks like **`checkPermission`**.

**Vulnerable Indication**

- No reference to user identity in the update/delete logic.
- Only ID is checked, not ownership or role.

**Secure Indication**

- Ownership check comparing object owner to logged-in user.
- Role-based privilege allowing authorized overrides.

### Vulnerable Code Example

```
// Vulnerable: Any user can delete any comment if ID is known
@DeleteMapping("/comments/{id}")
public ResponseEntity<?> deleteComment(@PathVariable Long id) {
    // Vulnerable function: deletes directly by ID
    commentRepo.deleteById(id);
    return ResponseEntity.ok("Deleted");
}
```

### Secure Code Example

```
// Secure: Only owner or admin can delete the comment
@DeleteMapping("/comments/{id}")
public ResponseEntity<?> deleteComment(@PathVariable Long id, Principal principal) {
    Comment comment = commentRepo.findById(id).orElseThrow();

    // Secure indication: Ownership validation with admin override
    if (!comment.getOwnerId().equals(principal.getName()) &&
!authzService.isAdmin(principal)) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
    }
    commentRepo.delete(comment); // Secure function with validation
    return ResponseEntity.ok("Deleted");
}
```

## 2.3. Missing Authorization on Endpoint

**Where to Check**

- Route declarations in Express (**`app.get`**, **`router.post`**).
- Controller mappings in Spring (**`@RequestMapping`**, **`@GetMapping`**).
- Admin routes or sensitive data endpoints.

**What to Check**

- Is there middleware (**`ensureAuthenticated`**, **`requireLogin`**) protecting the route?
- Are role checks enforced (**`ensureRole('admin')`**)?
- Does the system return **`403 Forbidden`** when unauthorized?

**Vulnerable Functions**

- **`app.get()`**, **`router.post()`** without middleware.
- Static file serving (**`express.static()`**) exposing sensitive files.

**Secure Functions**

- Middleware enforcing authentication (**`ensureAuthenticated`**).
- Role-based annotations (**`@PreAuthorize("hasRole('ADMIN')")`**).

**Vulnerable Indication**

- No middleware or security annotations before route handler.
- Endpoint accessible without login.

**Secure Indication**

- Middleware exists to validate session.
- Role-based restriction is applied before serving data.

### Vulnerable Code Example

```
// Vulnerable: Admin dashboard open to public
app.get('/admin/dashboard', (req, res) => {
  // Vulnerable function: serves sensitive file directly
  res.sendFile(__dirname + "/admin-dashboard.html");
});
```

### Secure Code Example

```
// Secure: Protect endpoint with authentication & role checks
app.get('/admin/dashboard',
  ensureAuthenticated,  // Secure function: verifies login
  ensureRole('admin'),  // Secure function: verifies admin role
  (req, res) => {
    res.sendFile(__dirname + "/admin-dashboard.html");
  }
);
```

## 2.4. Forceful Browsing

**Where to Check**

- Static file serving (**`express.static`**).
- File system resource mapping (**`sendFile`**, **`FileSystemResource`**).
- Hidden paths (e.g**., `/backup.zip`, `/admin/`**).

**What to Check**

- Are sensitive resources protected by authentication middleware?
- Are role checks present before serving files?
- Are directory indexes disabled?

**Vulnerable Functions**

- **`express.static()`** exposing sensitive folders.
- **`sendFile()`** used without authorization checks.

**Secure Functions**

- **`express.static()`** used with **`ensureAuthenticated`.**
- Resource handlers wrapped with role checks.

**Vulnerable Indication**

- Resource mapped directly to a URL without checks.
- Admin pages or logs accessible without login.

**Secure Indication**

- Middleware ensures only authenticated/authorized users can access.
- Role-based checks explicitly required before serving files.

### Vulnerable Code Example

```
// Vulnerable: Admin panel exposed publicly
app.use('/admin', express.static('admin-panel'));
// Vulnerable indication: No authentication or role checks
```

### Secure Code Example

```
// Secure: Protect admin panel with authentication & role checks
app.use('/admin',
  ensureAuthenticated, // Secure function: enforces login
  ensureRole('admin'), // Secure function: enforces admin role
  express.static('admin-panel') // Secure indication: access only after validation
);
```

## 2.5. Improper RBAC (Role-Based Access Control)

**Where to Check**

- Authorization middleware/annotations.
- Controller methods that differentiate user roles.
- Business logic functions that restrict access by role.

**What to Check**

- Are sensitive operations role-restricted (e.g., only admins can delete users)?
- Are role checks consistent across the codebase?
- Is role validation bypassable through missing middleware or direct function calls?

**Vulnerable Functions**

- Route handlers without role checks.
- Service methods accepting requests from any user role.

**Secure Functions**

- Role-based middleware like **`ensureRole("admin")`**.
- Security annotations **(`@RolesAllowed`**, **`@PreAuthorize("hasRole('ADMIN')")`).**

**Vulnerable Indication**

- Code only checks if user is authenticated, but not their role.
- Privileged operations (delete, modify users, approve payments) accessible to regular users.

**Secure Indication**

- Role validation is explicitly present.
- Least privilege is enforced (only required roles have access).

### Vulnerable Code Example

```
//  Vulnerable: Any logged-in user can delete users
app.delete('/users/:id', ensureAuthenticated, async (req, res) => {
  await db.users.deleteById(req.params.id); // No role check
```

```
  res.send("User deleted");
});
```

### Secure Code Example

```
// Secure: Only admins can delete users
app.delete('/users/:id',
  ensureAuthenticated,        // Secure function: verifies login
  ensureRole('admin'),        // Secure function: validates role
  async (req, res) => {
    await db.users.deleteById(req.params.id);
    res.send("User deleted");
```

## 2.6. BFLA (Broken Function-Level Authorization)

**Where to Check**

- API endpoints providing multiple functions for same resource.
- Frontend-to-backend function calls (`approve`, `publish`, `delete`).
- Controllers where different methods are exposed to all authenticated users.

**What to Check**

- Is there **authorization enforcement per function**?
- Are sensitive actions (e.g., publish, approve, delete) restricted to correct roles?
- Does the frontend hide buttons, but backend still allows access?

**Vulnerable Functions**

- **`deleteById()`**, **`approvePost()`**, **`publishArticle()`** exposed without role checks.

**Secure Functions**

- Function-level authorization via **`@PreAuthorize`**, **`checkPermission`**, or middleware.

**Vulnerable Indication**

- Endpoint protected only by login, no role/function-level checks.
- Backend trusts UI to hide restricted actions.

**Secure Indication**

- Role/function-level restriction explicitly present in backend.

### Vulnerable Code Example

```
//  Vulnerable: Any authenticated user can approve posts
@PostMapping("/posts/{id}/approve")
public ResponseEntity<?> approvePost(@PathVariable Long id) {
    Post post = postRepo.findById(id).orElseThrow();
    post.setApproved(true);
    postRepo.save(post); // No function-level authorization
    return ResponseEntity.ok("Approved");
```

### Secure Code Example

```
// Secure: Only moderators/admins can approve posts
```

```java
@PostMapping("/posts/{id}/approve")
@PreAuthorize("hasRole('ADMIN') or hasRole('MODERATOR')") // Secure function-level
authorization
public ResponseEntity<?> approvePost(@PathVariable Long id) {
    Post post = postRepo.findById(id).orElseThrow();
    post.setApproved(true);
    postRepo.save(post);
    return ResponseEntity.ok("Approved");
```

## 2.7. Weak Access Permissions

**Where to Check**

- File system permission configurations.
- Database queries fetching multiple users' data.
- ACL (Access Control List) checks.

**What to Check**

- Are permissions correctly scoped (least privilege)?
- Are sensitive resources exposed by default to all users?
- Are ACLs enforced consistently?

**Vulnerable Functions**

- **`SELECT * FROM users`** returning all users to a normal user.
- **`fs.readFile()`** exposing sensitive files.

**Secure Functions**

- **`findByOwnerId()`** ensuring access only to user's own data.
- Explicit role-based access checks for privileged data.

**Vulnerable Indication**

- Overly broad database queries.
- All authenticated users accessing resources meant for admins only.

**Secure Indication**

- Query scoped to current user.
- Role checks guarding sensitive resources.

## 2.8. Mass Assignment

**Where to Check**

- Model binding (e.g., **`req.body`** mapped directly to database).
- ORM usage (**`update(user, req.body)`**).

**What to Check**

- Are sensitive fields (**`role`**, **`isAdmin`**, **`permissions`**) filtered out?
- Is whitelist/allowlist applied for safe fields?

**Vulnerable Functions**

- **`Object.assign(user, req.body)`**
- ORM auto-binding (**`Model.update(req.body)`**)

**Secure Functions**

- Whitelisted fields: `pick(req.body, ["name","email"])`.
- Explicit property mapping.

**Vulnerable Indication**

- Direct mapping of request body to database model.
- No filtering of sensitive fields.

**Secure Indication**

- Explicit whitelist of safe fields for update.
- Sensitive fields ignored or protected by backend.

### Vulnerable Code Example

```
// Vulnerable: Attackers can promote themselves to admin
app.post('/users/update', ensureAuthenticated, async (req, res) => {
  const user = await db.users.findById(req.user.id);
  Object.assign(user, req.body); // Mass assignment vulnerability
  await user.save();
  res.json(user);
```

### Secure Code Example

```
// Secure: Whitelist allowed fields only
app.post('/users/update', ensureAuthenticated, async (req, res) => {
  const user = await db.users.findById(req.user.id);

  // Secure indication: Only name and email can be updated
  const allowed = (({ name, email }) => ({ name, email }))(req.body);
  Object.assign(user, allowed);

  await user.save();
  res.json(user);
```

## 2.9. User Enumeration

**Where to Check**

- Authentication flows (`login`, `signup`, `forgot password`).
- Error messages returned by API or UI.

**What to Check**

- Do error messages differ between valid and invalid users?
- Is timing response different for existing vs non-existing accounts?
- Is account lockout or rate limiting enforced?

**Vulnerable Functions**

- `res.send("User not found")` on login.
- Different responses for invalid vs valid usernames.

**Secure Functions**

- Generic error messages (**`Invalid username or password`**).
- Unified response regardless of username existence.

**Vulnerable Indication**

- Different error messages for invalid usernames vs wrong password.
- Timing differences based on lookup.

**Secure Indication**

- Uniform error responses.
- Logging done internally without exposing to user.

### Vulnerable Code Example

```
// Vulnerable: Reveals whether user exists
app.post('/login', async (req, res) => {
  const user = await db.users.findByEmail(req.body.email);
  if (!user) {
    return res.status(404).send("User does not exist"); // Enumeration leak
  }
  if (!(await user.verifyPassword(req.body.password))) {
    return res.status(401).send("Incorrect password");
  }
  res.send("Login successful");
```

### Secure Code Example

```
// Secure: Generic error messages prevent enumeration
app.post('/login', async (req, res) => {
  const user = await db.users.findByEmail(req.body.email);

  // Secure indication: Same response whether user exists or not
  if (!user || !(await user.verifyPassword(req.body.password))) {
    return res.status(401).send("Invalid username or password");
  }
  res.send("Login successful");
```

## 3. Cryptographic Failures

### 3.1 Weak Hashing Algorithms (MD5, SHA1, etc)

- **What to review:** **`MessageDigest.getInstance("MD5"|"SHA-1")`**, **`crypto.createHash('md5')`**, **`Digest::MD5`**, any use of MD5/SHA1 for password/storage/integrity.
- **Where:** hashing utility modules, legacy helper functions, database migration scripts.
- **How:** ensure only collision-resistant functions used; if hashing for passwords use KDFs (see 3.6). For non-password use SHA-256+ or SHA-3 with HMAC if integrity needed.

**Vulnerable Code Example:**

```
MessageDigest md = MessageDigest.getInstance("MD5");
byte[] digest = md.digest(data);
```

**Secure Code Example:**

```
MessageDigest md = MessageDigest.getInstance("SHA-256"); // for non-password digests
```

- **Vulnerable indicators:** `MD5`, `SHA1`, `crc32`, `md5sum` usage in app logic.
- **Secure indicators:** `SHA-256`, `SHA-512`, `SHA3-256`, `HMAC-SHA256`, use of HMAC when integrity is required

## 3.2 No Salting for password hashing

- **What to review:** raw `hash(password)`, `DigestUtils.sha256(password)` ; code that stores deterministic hash.
- **Where:** user auth modules, registration/login handlers, migration scripts.
- **How:** verify KDF usage – Argon2 / bcrypt / scrypt / PBKDF2. Salt generation should be per-user and stored with the hash (or use built-in salt of bcrypt).

**Vulnerable Code Example:**

```
stored = sha256(password.encode()).hexdigest()
```

**Secure Code Example:**

```
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())  # salt embedded in result
```

- **Vulnerable indicators:** hashing functions applied directly to `password` variable; no salt field in DB.
- **Secure indicators:** `bcrypt.hashpw`, `argon2.PasswordHasher`, `PBKDF2` with `salt` parameter stored per user.

## 3.3 Hardcoded credentials & secrets leakage

- **What to review:** literal strings like `API_KEY = "..."`, `private_key` content, `-----BEGIN PRIVATE KEY-----`.
- **Where:** source files, `resources/`, `secrets/`, CI pipeline files (`.github/workflows`, `.gitlab-ci.yml`), config templates, Dockerfiles.
- **How:** grep for suspicious patterns, check git history for prior commits, check tests/examples for test secrets.

**Vulnerable Code Example:**

```
const STRIPE_KEY = "sk_live_ABC...";
```

**Secure Code Example:**

```
const STRIPE_KEY = process.env.STRIPE_KEY; // load from env or secrets manager
```

- **Vulnerable indicators:** long base64 strings, `AKIA`, `ssh-rsa`, `BEGIN PRIVATE KEY`, `password=` in code.
- **Secure indicators:** `process.env.*`, `System.getenv`, `vault.getSecret()`, references to KMS/Vault/Secrets Manager.

## 3.4 Insecure Key Management

- **What to review:** where keys are stored/loaded, key rotation logic, key length/format.
- **Where:** config, deployment manifests, key-loading functions, key-store connectors.
- **How:** confirm keys come from secret manager / environment / HSM; check code for rotation, key id mapping, and storage permissions.

**Vulnerable Code Example:**

```
Key key = loadFromFile("/etc/app/keys/mykey.pem"); // in repo or world-readable
```

- **Secure:** keys obtained from a managed secret store (AWS KMS, GCP KMS, Azure Key Vault) or hardware-backed store; enforce rotation policy.

- **Vulnerable indicators: keys** in repo, **`readFile("secrets")`**, **`chmod 644`** files.

- **Secure indicators:** **`SecretsManagerClient`**, **`getSecretValue`**, **`KeyManagementService`**, **`HSM`**, IAM policy check.

## 3.5 Insecure Randomness

- **What to review:** use of **`Math.random()`**, **`new Random()`** (non-secure), **`UUID`** usage assumptions.
- **Where:** token generation, nonce/IV, key generation code.
- **How:** ensure CSPRNG used: **`SecureRandom`** (Java), **`secrets`** (Python), **`crypto.randomBytes`** (Node), **`window.crypto.getRandomValues`** (browser).

**Vulnerable Code Example:**

```
const token = Math.random().toString(36).slice(2);
```

**Secure Code Example:**

```
const token = crypto.randomBytes(32).toString('hex');
```

- **Vulnerable indicators:** **`Math.random`**, **`Random()`** without Secure suffix, time-seed manipulations.

- **Secure indicators:** **`SecureRandom`**, **`crypto.randomBytes`**, **`secrets.token_bytes`**, **`getRandomValues`**.

## 3.6 Improper password storage

- **What to review:** storing plaintext or reversible encryption for passwords.
- **Where:** user table insertion, backup scripts, SSO connectors.
- **How:** ensure one-way salted hashing + slow KDF; ensure migration scripts that convert legacy storage are safe (do not store intermediate plaintext).

**Vulnerable Code Example:**

```
INSERT INTO users (password) VALUES ('plain-text');
```

**Secure Code Example:**

```
INSERT INTO users (password) VALUES ('plain-text');
```

- **Secure:** **`INSERT ... VALUES (argon2_hash`**) and verify on login with **`argon2.verify`**.

- **Vulnerable indicators:** `encrypt(password)` used to "store" passwords, `BASE64` encoding passwords.
- **Secure indicators:** `bcrypt`, `argon2`, `PBKDF2` usage with per-user salt.

## 3.7 Improper use of encryption modes

- **What to review:** AES-CBC used without MAC, manual IV reuse.
- **Where:** crypto utils, file encryption, database encryption libraries.
- **How:** prefer AEAD (AES-GCM, ChaCha20-Poly1305). If using CBC, verify Encrypt-then-MAC is applied and IV is random & unique.

**Vulnerable Code Example:**

```
cipher = AES.new(key, AES.MODE_CBC, iv)
ciphertext = cipher.encrypt(pad(plaintext))
```

**Secure Code Example:**

```
cipher = AES.new(key, AES.MODE_GCM)
ciphertext, tag = cipher.encrypt_and_digest(plaintext)
```

- **Vulnerable indicators:** `AES.MODE_CBC`, `CBC` without `HMAC`.
- **Secure indicators:** `GCM`, `ChaCha20-Poly1305`, presence of `tag` or `auth` verification.

## 3.8 Insecure protocol usage

- **What to review:** disabling TLS verification, use of plain HTTP for sensitive APIs, old TLS version forcing.
- **Where:** HTTP client code, server TLS configuration (SSLContext), 3rd-party connectors.
- **How:** ensure `verify=True` (Python requests), hostname verification enabled (Java), TLS >= 1.2/1.3, not forcing TLSv1 or SSLv3.

**Vulnerable Code Example:**

```
requests.get(url, verify=False)
```

Secure Code Example:

```
requests.get(url, verify='/path/to/ca-bundle.pem')
```

- **Vulnerable indicators:** `verify=False`, custom TrustManager that returns true, `SSLContext.setProtocol("TLSv1").`
- **Secure indicators:** default TLS stacks, explicit TLSv1.2+/TLSv1.3 minimum, proper truststore usage.

## 3.9 Lack of digital signatures or integrity protection

- **What to review:** files or messages transmitted/stored without integrity checks.
- **Where:** backups, firmware update checks, signed tokens, file storage.

- **How:** ensure HMAC or digital signatures (RSA/ECDSA) protect integrity and authenticity; verify signature before use.

**Vulnerable Code Example:**

```
accept_file(plain_file)
```

- **Secure:** **`verify_signature(public_key, file, signature)`** before processing.
- **Vulnerable indicators:** absence of **`verify`**/**`hmac.compare_digest`**.
- **Secure indicators:** explicit **`HMAC`**, **`verify`**, **`signature`** checks using crypto libraries.

## 3.10 Replay and session token issues

- **What to review:** tokens without `exp`, no `jti` or nonce checks, long-lived session ids.
- **Where:** JWT handling, session store, auth middleware.
- **How:** require `exp` and short lifetimes for access tokens, use refresh tokens, enforce `jti` uniqueness, invalidation on logout.

**Vulnerable Code Example:**

```
// token accepted indefinitely
if (verifySignature(token)) grantAccess();
```

**Secure Code Example:**

- **Secure:** check **`exp`**, **`nbf`**, **`jti`** uniqueness; store used **`jti`** for replay prevention.
- **Vulnerable indicators:** tokens accepted if signature OK even if old; `exp` missing.
- **Secure indicators:** **`exp`** checks, **`jti`** DB, session revocation endpoints.

## 3.11 Cryptographic padding issues

- **What to review:** manual padding/unpadding code, error messages leaking padding failures.
- **Where:** decryption paths, IO handlers, protocol parsers.
- **How:** avoid manual padding checks; use AEAD. If CBC with PKCS#7 is used, ensure decryption error handling doesn't reveal oracle information (same error for all failures).
- **Vulnerable:** manual unpad returning different errors that leak information.
- **Secure:** use **`AES-GCM`** / **`ChaCha20-Poly1305.`** When using CBC, implement encrypt-then-MAC.
- **Vulnerable indicators:** **`unpad()`**, **`PaddingException`** thrown and detailed to client.
- **Secure indicators:** AEAD libs, uniform error messages on crypto failure.

## 3.12 Sensitive data exposure in logs (secrets leakage)

- **What to review:** logger calls with tokens/keys/passwords, stack traces that include secrets.
- **Where:** exception handlers, request/response logging, debug prints, CI logs.
- **How:** find **`logger.info`**/**`debug`**/**`error`** concatenations with variables named token/password/key; design redaction functions; ensure production debug off.

**Vulnerable Code Example:**

```
logger.info("Login failed for user " + username + ", token=" + token);
```

**Secure Code Example:**

```
logger.info("Login failed for user {}", username); // token omitted or redacted
```

- **Vulnerable indicators:** `println`, `console.log` dumps of `req.headers`, `Authorization`.
- **Secure indicators:** redaction functions, `MASKED`, scrubbed logs, structured logging with redaction rules.

## 3.13 Insufficient entropy

- **What to review:** seed usage from low-entropy sources, generation in constrained environments/containers without entropy seeding.
- **Where:** build containers, VM images, token generation.
- **How:** ensure CSPRNG uses OS entropy; for containers ensure `/dev/urandom` available; avoid deterministic seeds sourced from timestamps or predictable values.
- **Vulnerable:** `Random r = new Random(12345);`
- **Secure:** `SecureRandom sr = SecureRandom.getInstanceStrong();`
- **Vulnerable indicators:** explicit seeds based on time, tiny entropy pools in tests being used in prod.
- **Secure indicators:** `getInstanceStrong`, `secrets` or `/dev/urandom` usage.

## 3.14 Weak cipher suite enforced in code setting

- **What to review:** `setEnabledCipherSuites([...weak...])`, server TLS config forcing RSA key exchange or RC4.
- **Where:** Netty/OkHttp/SSLContext/SSLSocketFactory configuration.
- **How:** ensure ECDHE key exchange + AEAD ciphers permitted; remove RC4, DES, 3DES, NULL ciphers.

**Vulnerable Code Example:**

```
sslEngine.setEnabledCipherSuites(new String[]{"TLS_RSA_WITH_AES_128_CBC_SHA"});
```

- **Secure:** enable `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256` or **TLS 1.3** default suites.
- **Vulnerable indicators:** explicit listing of `CBC` suites, `RC4`, `DES`.
- **Secure indicators:** ECDHE, GCM, TLSv1.3 usage.

## 3.15 Use of base64 as encryption

- **What to review:** code that treats base64 encoding as "encryption".
- **Where:** older helper functions, lightweight obfuscation helpers.
- **How:** check for `Base64.encode` used instead of real crypto; ensure encoding is not mistaken for confidentiality.

**Vulnerable Code Example:**

```
const cipher = Buffer.from(plaintext).toString('base64'); // NOT encryption
```

- **Secure:** perform real encryption **`crypto.createCipheriv(...).update(...)`**.
- **Vulnerable indicators:** comments saying "encoded" == "encrypted", base64 used for secrets.
- **Secure indicators:** use of AES-GCM/ChaCha20, proper IV and tag handling.

## 3.16 JWT with 'none' algorithms accepted

- **What to review:** JWT verification logic that trusts the **`alg`** header or ignores verification.
- **Where:** token validation middleware.
- **How:** ensure library verifies signature and rejects **`alg: none`**. Do not implement manual verify that only decodes payload.

**Vulnerable Code Example:**

```
const payload = jwt.decode(token); if (payload) grantAccess(); // no verify
```

**Secure Code Example:**

```
jwt.verify(token, publicKey, {algorithms: ['RS256']});
```

- **Vulnerable indicators:** use of **`decode`** without **`verify`**, acceptance **of `alg: 'none'`**.
- **Secure indicators:** explicit **`verify()`** calls, algorithm whitelist.

## 3.17 JWT signed with weak secret (short key)

- **What to review: HS256** secrets that are short/simple or stored in repo.
- **Where:** auth config, environment variables, secrets store.
- **How:** check secret lengths; for HMAC choose ≥ 32 bytes of high entropy; prefer asymmetric (RS256/ES256) for distributed systems.
- **Vulnerable: `JWT_SECRET = "abc123"`**
- **Secure:** random 256-bit secret from vault or use RSA keypair.
- **Vulnerable indicators:** short strings, human-readable passphrases.
- **Secure indicators:** long base64/random secrets, references to private/public keys.

## 3.18 JWT without expiration

- **What to review:** tokens with missing/very long **`exp`**.
- **Where:** token issuer code, refresh token flows.
- **How:** ensure `exp` present and reasonable (minutes/hours for access, longer for refresh with rotation), enforce rejection if missing.
- **Vulnerable:** token created without **`exp`**.
- **Secure:** create token with **`exp: now + short_lifetime.`**
- **Vulnerable indicators: `expiresIn: null`**, absence of **`exp`** in token payload.
- **Secure indicators: `exp`, `nbf`**, short lifetime + refresh flow.

## 3.19 Improper signature verification

- **What to review:** custom signature checks (string equality), not using constant-time compare.
- **Where:** signature verification functions, JWT handling, HMAC validation.
- **How:** use library **verify** functions; when comparing digests, use constant-time comparators to avoid timing attacks.

**Vulnerable Code Example:**

```
if computed_signature == provided_signature: accept()
```

**Secure Code Example:**

```
hmac.compare_digest(computed_signature, provided_signature)
```

- **Vulnerable indicators:** == comparisons for MACs, manual base64 decode + compare.
- **Secure indicators:** `verify()` APIs, `compare_digest`.

## 3.20 JWT KID Injection

- **What to review:** selecting verification key directly from `kid` header without validating that key is trusted for this issuer.
- **Where:** key selection code, JWKS retrieval handlers.
- **How:** map `kid` to known keys and ensure key material is fetched from trusted JWKS/keystore; never accept arbitrary KID pointing to attacker key.

**Vulnerable Code Example:**

```
key = fetch(req.header.kid_url); verify(token, key)
```

- **Secure:** use trusted JWKS endpoint + verify **`iss`**/**`aud`** and that KID exists in your trusted set.
- **Vulnerable indicators:** dynamic fetching of keys from header values, no issuer/audience checks.
- **Secure indicators:** static jwks cache from trusted provider, KID whitelisting.

## 3.21 No Secure flag in cookie

- **What to review:** cookie creation lines in server code (Set-Cookie).
- **Where:** auth cookie setup, session creation.
- **How:** ensure `Secure` added on cookies containing session IDs or tokens when served over HTTPS.

**Vulnerable Code Example:**

```
Set-Cookie: session=abc; Path=/; HttpOnly
```

**Secure Code Example:**

```
Set-Cookie: session=abc; Path=/; Secure; HttpOnly; SameSite=Lax
```

- **Vulnerable indicators:** absence of `Secure` in cookie strings.
- **Secure indicators:** `Secure` flag present.

### 3.22 No HttpOnly flag in cookie

- **What to review:** cookie headers or cookie builder APIs.
- **Where:** auth/session code.
- **How:** ensure `HttpOnly` set to prevent JS access to session cookies.
- **Vulnerable:** cookie created without `HttpOnly`.
- **Secure:** cookie builder sets `HttpOnly=true`.
- **Vulnerable indicators:** `document.cookie` used for session cookie set, `Set-Cookie` missing HttpOnly.
- **Secure indicators:** `HttpOnly` present.

### 3.23 Missing SameSite attribute in cookie

- **What to review:** cookie attributes.
- **Where:** auth/session cookie creation.
- **How:** use `SameSite=Lax` or `Strict` for session cookies, `None` only with Secure and for cross-site use-cases.
- **Vulnerable:** `Set-Cookie: session=abc; Path=/; Secure; HttpOnly` (no SameSite)
- **Secure:** `Set-Cookie: session=abc; Path=/; Secure; HttpOnly; SameSite=Strict`
- **Vulnerable indicators:** missing SameSite header on session cookies.
- **Secure indicators:** SameSite explicitly set to `Lax` or `Strict`.

## 4. Input Validation & Sanitization (Injections)

## 4.1) Lack of Input Validation & Sanitization

**Where to check**

- API controllers, route handlers, form processors, file upload endpoints, query parsers, config parsing code, CLI argument parsers, inter-service message handlers.

**What to check**

- Inputs accepted from users (HTTP params, JSON bodies, headers, cookies, form fields, file names) that are used without validation.
- Business logic that assumes inputs are safe (length, type, charset, format).
- Missing centralized validation layer / reliance on client-side validation only.

**How to check**

- Review input handling code paths: follow the flow from entry point → business logic → sinks (DB, filesystem, OS calls, template rendering, network calls).
- Check for absence of validation libraries (e.g., missing schema validation like JSON Schema, Joi, javax.validation).
- Verify presence of whitelisting (allow-list) vs blacklisting.
- Dynamic: send malformed or type-mismatched inputs in a safe test environment to see errors/behavior; monitor logs for unhandled exceptions.

**Vulnerable patterns**

- Direct use of request parameters in logic without checks.
- `if (param != null) use(param)` without type/format/length checks.

**Secure patterns**

- Input canonicalization + centralized schema validation (JSON Schema, DTOs with annotations, Joi, Marshmallow).
- Use allow-lists for accepted values, strict length/charset checks, and reject-or-log unknown fields.

**Search keywords**

- `getParameter`, `request.body`, `req.params`, `input()`, `argparse`, `@RequestBody` (Java), `req.body` (Node), `isset($_POST)`, `validate`, `schema`, `@Valid`, `Joi`, `jsonschema`.

## Vulnerable Code Example:

```java
// Servlet example: no validation
// read "id" parameter from HTTP request (untrusted input)
String id = request.getParameter("id");
// parse to int without checking format -> NumberFormatException or injection of
unexpected value
int userId = Integer.parseInt(id);
// use parsed value to fetch user (trusts previous parsing)
User u = userService.getUserById(userId);
// output user name without escaping -> potential XSS if name contains markup
out.println("Hello " + u.getName());
```

**Secure Code Example:**

```java
// Validate and canonicalize at boundary
// read "id" from request
String id = request.getParameter("id");
// ensure id is numeric and within length bounds (allow-list)
if (id == null || !id.matches("^[0-9]{1,10}$")) {
   // return 400 on invalid input
    response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid id");
    return; // stop processing on invalid input
}
// safe to parse because validated
int userId = Integer.parseInt(id);
// fetch user safely
User u = userService.getUserById(userId);
// escape HTML to prevent XSS
out.println(org.apache.commons.text.StringEscapeUtils.escapeHtml4(u.getName()));
```

## 4.2 Weak Input Validation & Sanitization

**Where to check**

- Same places as (1), plus third-party integrations where inputs are transformed.

**What to check**

- Validation present but permissive: e.g., regex allowing dangerous characters, numeric checks done by parsing without range checks, insufficient length limits, Unicode normalization missing.

**How to check**

- Inspect regexes and validators for overly-broad patterns (e.g., `.*`, `^[\w\W]+$`).
- Confirm Unicode normalization and encoding handling (NFC/NFD), leading/trailing whitespace handling.
- Test boundary values, extreme lengths, special character sets, null/empty vs missing fields.

**Vulnerable patterns**

- Regexes that are trivially bypassable.
- Relying on `trim()` or `isNumeric()` as the only checks.
- Accepting file extensions from client without verifying content.

**Secure patterns**

- Strict allow-list validators, canonicalization before validation, enforce types at the boundary (use typed DTOs), validate decoded values (after URL/HTML decode).

**Search keywords**

- `regex`, `pattern`, `trim`, `parseInt`, `is_numeric`, `toString()`, `validateEmail`, `length >`, `maxLength`, `normalize`.

### Vulnerable Code Example:

```
 // Servlet example: no validation
// read "id" parameter from HTTP request (untrusted input)
String id = request.getParameter("id");
// parse to int without checking format -> NumberFormatException or injection of
unexpected value
int userId = Integer.parseInt(id);
// use parsed value to fetch user (trusts previous parsing)
User u = userService.getUserById(userId);
// output user name without escaping -> potential XSS if name contains markup
out.println("Hello " + u.getName());
```

### Secure Code Example:

```
// Validate and canonicalize at boundary
// read "id" from request
String id = request.getParameter("id");
// ensure id is numeric and within length bounds (allow-list)
if (id == null || !id.matches("^[0-9]{1,10}$")) {
   // return 400 on invalid input
    response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid id");

    return; // stop processing on invalid input
}
// safe to parse because validated
int userId = Integer.parseInt(id);
// fetch user safely
User u = userService.getUserById(userId);
// escape HTML to prevent XSS
out.println(org.apache.commons.text.StringEscapeUtils.escapeHtml4(u.getName()));
```

## 4.3) Cross-Site Scripting (XSS) — Reflected, Stored, DOM

**Where to check**

- Template rendering code, client-side DOM manipulation, innerHTML/outerHTML assignments, server responses echoing user input (search results, error messages), comment areas, profile fields, rich-text inputs.

**What to check**

- Output contexts: HTML body, attributes, JavaScript contexts, CSS, URL context, and JSON used in scripts.
- Whether server-side templating auto-escapes outputs; whether client code inserts HTML without sanitization.
- Rich text editors that accept HTML (sanitize?).

**How to check**

- Review server-side templates for raw output (e.g., `{{ raw }}`, `@Html.Raw`, `innerHTML`).
- In client JS, search for direct DOM insertion functions (`innerHTML`, `outerHTML`, `insertAdjacentHTML`, `document.write`).
- Check CSP headers and their strictness.
- Verify libraries used for sanitization are configured correctly (allow-lists vs remove-only).
- Test: put harmless markup-like strings and check they render as text not markup in different contexts (in safe test environments only).

**Vulnerable patterns / functions**

- Server: `@Html.Raw`, template tag that disables escaping, building HTML strings with `+`.
- Client: `element.innerHTML = userInput`, `eval()` used to process returned templates.
- Rich text stored without sanitization.

**Secure patterns**

- Contextual output encoding: HTML-encode for element content, attribute-encode for attributes, JS-escape for inside scripts, URL-encode for redirect parameters.
- Use secure templating engines with auto-escaping and avoid raw output.
- Use vetted sanitizers (DOMPurify with strict config) *and* canonicalize before sanitizing.

**Search keywords**

- `innerHTML`, `outerHTML`, `document.write`, `@Html.Raw`, `dangerouslySetInnerHTML`, `raw`, `unescaped`, `DOMPurify`, `sanitize`, `escapeHtml`, `setAttribute(value)`.

## Vulnerable Code Example:

```
 <!-- read comment parameter from request (untrusted) -->
 String comment = request.getParameter("c");
<!-- directly inject comment into HTML without escaping -> reflected XSS -->
<p>User comment: <%= comment %></p>
```

## Secure Code Example:

```
// escape HTML chars
String safe = org.apache.commons.text.StringEscapeUtils.escapeHtml4(request.getParameter("c"));
// output escaped value -> prevents XSS
out.println("<p>User comment: " + safe + "</p>");
```

## 4.4. SQL Injection (Error, Boolean-based blind, Time-based, First & Second-order)

**Where to check**

- All DB access code: raw SQL queries, dynamic query builders, ORM raw query paths, logging of SQL strings, migrations that build SQL from inputs.

**What to check**

- Any SQL assembled using string concatenation or interpolation with user input.
- Parameter binding usage in prepared statements/ORM query APIs.
- Locations where user input is stored and later used in queries (second-order).

**How to check**

- Static: search for string concatenation with query text and variables.
- Check DB access layer for use of APIs that support parameterization and whether they are used.
- Look for logging of SQL that may leak structure/data.
- Dynamic testing (defensive): use safe inputs that include special SQL characters to confirm queries use parameters (observe behavior/response), analyze error messages returned by DB for richness.

**Vulnerable functions / patterns**

- String concatenation to form SQL.
- `execute("SELECT ... " + param)`, `String.format` with user input.
- ORM raw SQL execution with interpolated strings.

**Secure patterns / functions**

- Use parameterized queries / prepared statements / query builders that bind parameters (JDBC PreparedStatement, psycopg2 with parameters, parameterized ORM methods).
- Use stored procedures with parameters when appropriate.
- For second-order SQLi: sanitize before storage and/or treat stored data as untrusted when used in queries.

**Search keywords**

- `execute(` + +, `format(` + `SELECT`, `rawQuery`, `queryBuilder`, `Statement.execute`, `createQuery("..."+`, `string interpolation` (backticks/format).

## Vulnerable Code Example:

```
// take username from request
String username = request.getParameter("user");
// build SQL Query via string concatenation -> SQLi
String sql = "SELECT * FROM users WHERE username = '" + username + "'";
// create a Statement (not parameterized)
Statement s = connection.createStatement();
// execute raw SQL Query containing user input -> vulnerable
ResultSet rs = s.executeQuery(sql);
```

## Secure Code Example:

```
// get username
String username = request.getParameter("user");
// parameterized SQL statement
String sql = "SELECT * FROM users WHERE username = ?";
// create prepared statement
try (PreparedStatement ps = connection.prepareStatement(sql)) {
    // bind user input as parameter (prevents injection)
    ps.setString(1, username);
    // execute query safely
    try (ResultSet rs = ps.executeQuery())
```

# 4.5. LFI, RFI & Directory Traversal

**Where to check**

- File read APIs, include/require calls (PHP), file upload handlers, path join logic, downloads, template include paths, static file endpoints.

**What to check**

- Path construction using user input without canonicalization.
- Use of `open()` / `FileInputStream` / `include` with user-controlled path.
- Web server config that maps to filesystem without filtering.

**How to check**

- Inspect path normalization and validation (no `..`, no absolute paths unless allowed).
- Confirm realpath/canonicalize and allow-listing of root directories.
- Check for null byte handling and OS path separator handling.
- Dynamic: in test environment, verify response to path-like inputs and ensure errors are normalized (don't leak file content in errors).

**Vulnerable patterns**

- `include($_GET['page'])`, `fs.readFile(path + user)`, concatenating user input into file paths.

**Secure patterns**

- Resolve to canonical path and check it starts with allowed base directory.
- Use allow-list mapping (e.g., map token → filename).
- Reject or sanitize path traversal characters before use.

**Search keywords**

- `include(`, `require(`, `FileInputStream(`, `open(` + `path`, `readFileSync`, `realpath`, `resolve`, `path.join` + user input.

### Vulnerable Code Example:

```
// user-controlled filename
String file = request.getParameter("file");
// naive concatenation allows "../" traversal
File f = new File("/var/www/data/" + file);
// opens potentially attacker-controlled file
FileInputStream fis = new FileInputStream(f);
```

### Secure Code Example:

```
// read filename
String file = request.getParameter("file");
// basic checks for traversal and null bytes
if (file == null || file.contains("..") || file.contains("\0")) {
    // reject suspicious input
    throw new SecurityException("Invalid filename");
}
// canonicalize base directory
Path base = Paths.get("/var/www/data").toRealPath();
// resolve and normalize requested path
Path resolved = base.resolve(file).normalize();
// ensure requested file is inside base directory
if (!resolved.startsWith(base)) {
    // reject traversal
    throw new SecurityException("Path traversal attempt");
}
// safely open file
try (InputStream in = Files.newInputStream(resolved))
```

## 4.6. NoSQL Injection

**Where to check**

- NoSQL query construction areas (MongoDB queries, CouchDB views, ElasticSearch queries), code building query objects from user input.

**What to check**

- Whether user input is inserted directly into query objects or query DSL (e.g., `$where` in Mongo).
- Use of string-based query builders that accept JSON or scripts.

**How to check**

- Review creation of query objects: ensure only expected keys are allowed and types are enforced.
- Check for use of `$where` or script/query execution features.
- Verify driver API is used correctly with parameter binding or typed queries.

**Vulnerable patterns**

- Passing entire decoded JSON from user into a query builder.
- Use of query languages that permit arbitrary script evaluation.

**Secure patterns**

- Whitelist queryable fields; enforce types; disallow operator injection (`$`-prefixed keys).

- Use safe query constructors and parameterization methods provided by drivers.

**Search keywords**

- `find(+{, $where, eval, search({, query_string, elastic+body, JSON.parse(req.body)` feeding to DB.

### Vulnerable Code Example:

```
// accept raw JSON query string from user
String queryJson = request.getParameter("q");
// parse into DB query object (user controls operators)
BasicDBObject query = (BasicDBObject) JSON.parse(queryJson);
// execute query containing user-supplied operators (e.g., $where) -> NoSQL injection
DBCursor cur = collection.find(query);
```

### Secure Code Example:

```
// get simple value from request
String name = request.getParameter("name");
// validate type/length
if (name == null || name.length() > 100) throw new IllegalArgumentException();
// construct safe filter using driver API (no operators injected)
Bson filter = Filters.eq("name", name);
// execute safe query
FindIterable<Document> docs = collection.find(filter);
```

## 4.7. XXE / XML Injection / XML Entity Expansion (XEE)

**Where to check**

- XML parsers (server-side), SOAP endpoints, SAML processors, RSS/Atom consumers, XML config loaders.

**What to check**

- Parser configuration: DTD/External entity resolution enabled by default in many libs.
- Use of insecure XML libraries or default configurations that resolve external entities.
- Uncontrolled use of `xml.parse` with user-supplied data.

**How to check**

- Verify parser features: ensure external entity resolution and DTDs are disabled.
- Check for use of lib-specific secure parsing options (e.g., disable DOCTYPE, set secure processing).
- Confirm size limits and recursion limits to mitigate entity expansion attacks.

**Vulnerable patterns**

- Using `DocumentBuilderFactory` without disabling external entities, or parsing XML directly into entity-resolving parsers.

**Secure patterns**

- Disable DTD and external entity resolution; use secure parser configuration or non-XML formats (JSON) when possible.
- Use XML sanitizer libraries or schema validation with safe parser settings.

**Search keywords**

- `DocumentBuilderFactory`, `XMLInputFactory`, `XMLParser`, `resolveEntity`, `DOCTYPE`, `ENTITY`, `external-entities`, `expandEntityReferences`.

**Vulnerable Code Example:**

```
// create XML parser factory
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
// create parser with default features
DocumentBuilder db = dbf.newDocumentBuilder();
// parse user XML; external entities may be resolved -> XXE
Document doc = db.parse(new InputSource(new StringReader(xmlFromUser)));
```

**Secure Code Example:**

```
// create factory
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
// disable DOCTYPE
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
// disable external general entities
dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);
// disable external parameter entities
dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
// disable XInclude
dbf.setXIncludeAware(false);
// don't expand entity refs
dbf.setExpandEntityReferences(false);
// safe parser configured
DocumentBuilder db = dbf.newDocumentBuilder();
// parse user XML safely
Document doc = db.parse(new InputSource(new StringReader(xmlFromUser)));
```

## 4.8. Server-Side Request Forgery (SSRF)

**Where to check**

- Code that fetches remote resources using URLs derived from user input: image fetchers, URL previewers, server-side HTTP clients, cloud metadata callers.

**What to check**

- Whether URLs from users are directly fetched.
- Lack of egress filtering and URL validation; no IP allow-list or DNS rebinding protections.
- Server attempts to access internal network (metadata endpoints) based on user-supplied hostnames.

**How to check**

- Inspect HTTP client usage for direct use of user URL.
- Ensure URL schemes restricted (disallow file://, gopher://, etc.), enforce DNS resolution checks, and block private IP ranges.
- Confirm outbound firewall / egress rules and host allow-lists exist.

**Vulnerable patterns**

- `httpClient.get(userProvidedUrl)` without validation.
- Following redirects blindly to internal addresses.

**Secure patterns**

- Parse and resolve host; block RFC1918/private IPs and metadata IPs; restrict allowed schemes and enforce hostname allow-list or allow-list by domain.
- Use server-side proxies that enforce egress policies.

**Search keywords**

- `HttpClient.get(+url, requests.get(url), fetch(url), allow_redirects, followRedirects, urlopen, openUrl.`

**Vulnerable Code Example:**

```
// user-supplied URL
String url = request.getParameter("url");
// open connection to arbitrary URL
HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
// set timeout
conn.setConnectTimeout(3000);
// fetch content from user-provided host -> SSRF risk (internal endpoints)
InputStream is = conn.getInputStream();
```

**Secure Code Example:**

```
// read URL
String urlStr = request.getParameter("url");
// parse URL
URL url = new URL(urlStr);
// extract host
String host = url.getHost();
// resolve host to IP
InetAddress addr = InetAddress.getByName(host);
// check for internal/private addresses
if (addr.isAnyLocalAddress() || addr.isSiteLocalAddress() ||
addr.isLoopbackAddress()) {
// block SSRF to internal services
    throw new SecurityException("Blocked internal IP");
}
// perform allow-list check
if (!isAllowedHost(host)) {
// block unknown hosts
    throw new SecurityException("Host not allowed");
}
// safe to open after checks
HttpURLConnection conn = (HttpURLConnection) url.openConnection();
// perform fetch with timeouts and content-type checks
```

## 4.9. Command Injection

**Where to check**

- Any place building OS commands from input: `exec`, `system`, shelling out in CI, scripting, spawn/exec in Node, ProcessBuilder, backticks in PHP.

**What to check**

- Concatenation of user input into shell commands; use of shell=True equivalents.
- Paths, filenames, or arguments passed to shells.

**How to check**

- Ensure code uses parameterized APIs that accept argument arrays (avoid shell execution).
- Review any use of system commands; verify inputs are sanitized/validated and not passed to a shell.
- Check for environment variable use and quoting.

**Vulnerable patterns**

- `system("cmd " + userInput)`, shell interpolation/backticks.

**Secure patterns**

- Use APIs that take argument arrays (ProcessBuilder, execFile), avoid shell parsing; escape/validate inputs strictly when shell needed.
- Use least-privileged service accounts and restrict allowed commands.

**Search keywords**

- `exec(`, `system(`, , `popen`, `Runtime.getRuntime().exec`, `child_process.exec`, `shell=True`.

## Vulnerable Code Example:

```
// user-supplied command fragment
String input = request.getParameter("cmd");
// executes a shell with user input concatenated -> command injection
Runtime.getRuntime().exec("sh -c " + input);
```

## Secure Code Example:

```
// get file argument
String arg = request.getParameter("file");
// validate allowed characters
if (arg == null || !arg.matches("^[A-Za-z0-9._-]{1,255}$")) throw new
IllegalArgumentException();
// pass arguments without shell interpolation
ProcessBuilder pb = new ProcessBuilder("/usr/bin/somecommand", arg);
// combine stdout/stderr
pb.redirectErrorStream(true);
// start process safely with arguments separated
Process p = pb.start();
```

# 4.10. Server-Side Template Injection (SSTI)

**Where to check**

- Template engines that evaluate expressions (Jinja2, Twig, Freemarker, Thymeleaf, Velocity), endpoints that accept templates or template fragments from users.

**What to check**

- Whether user input reaches template rendering contexts that evaluate expressions.
- Use of untrusted template strings passed into render functions.

**How to check**

- Inspect template rendering calls to ensure only data objects (not templates) are passed.
- Confirm templates are static and user inputs are bound as variables (escaped).
- Validate template engines configured to disable code execution features if possible.

**Vulnerable patterns**

- `render(templateFromUser)` or passing user-supplied string as template.
- Using templating APIs that execute expressions inside the template.

**Secure patterns**

- Never render user-supplied templates. Use strict template engines with no arbitrary code execution; treat templates as static, use sandboxing.

**Search keywords**

- `render(+template, Template("..."), freemarker, render_template_string, dangerouslySetInnerHTML` (React-esque context).

**Vulnerable Code Example:**

```
// rendering a template provided by user — DANGEROUS
// user provides template content
String templateFromUser = request.getParameter("tmpl");
// create FreeMarker config
Configuration cfg = new Configuration(Configuration.VERSION_2_3_31);
// build template from user input
Template t = new Template("userTpl", new StringReader(templateFromUser), cfg);
// render template; user can execute expressions -> SSTI
t.process(Map.of("data", userData), response.getWriter());
```

**Secure Code Example:**

```
// templates are static; user data passed as model only
// load pre-approved template
Template t = cfg.getTemplate("trustedTemplate.ftl");
// sanitize user data
Map<String,Object> model = Map.of("userInput", sanitize(userInput));
// render with data (template is not user-controlled)
t.process(model, response.getWriter());
```

## 4.11. LDAP Injection

**Where to check**

- LDAP query building code: authentication flows, group lookups, user search filters.

**What to check**

- Construction of LDAP filters with user input concatenated into filter strings.
- Use of unescaped characters ( `*`, `(`, `)`, `\` ).

**How to check**

- Check for raw filter string assembly using user input.
- Ensure escaping or prepared LDAP query APIs are used.
- Verify scopes and privileges for LDAP operations.

**Vulnerable patterns**

- `"(&(uid=" + username + ")(userPassword=" + pass + "))"`

**Secure patterns**

- Use LDAP escaping functions or parameterized LDAP libraries; validate and normalize inputs.

**Search keywords**

- `ldap`, `search(+filter`, `(&(uid=`, `ldapsearch`, `ldap.bind`.

**Vulnerable Code Example:**

```
// get uid from request
String uid = request.getParameter("uid");
// build LDAP filter via concatenation -> injection risk
String filter = "(&(objectClass=person)(uid=" + uid + "))";
// execute search with unsafe filter
```

```
NamingEnumeration<SearchResult> results = ctx.search("ou=people,dc=example,dc=com",
filter, sc);
```

**Secure Code Example:**

```
// read uid
String uid = request.getParameter("uid");
// escape LDAP special chars using encoder
String safeUid = LdapEncoder.filterEncode(uid);
// construct safe filter
String filter = "(&(objectClass=person)(uid=" + safeUid + "))";
// execute safe search
NamingEnumeration<SearchResult> results = ctx.search("ou=people,dc=example,dc=com",
filter, sc);
```

## 4.12. CRLF Injection

**Where to check**

- HTTP header construction, email generation, log writing, cookie building.

**What to check**

- Concatenation of user input into header values, mail headers, or log lines that allow CR (`\r`) or LF (`\n`) characters.

**How to check**

- Inspect header construction and ensure inputs are sanitized for new-line chars.
- Verify frameworks are not allowing raw header injection; test headers returned for unexpected line breaks.

**Vulnerable patterns**

- `response.setHeader("Location", "/?q=" + userInput)`

**Secure patterns**

- Remove or encode CR/LF characters before including in headers; use framework header APIs that validate values.

**Search keywords**

- `setHeader`, `addHeader`, `Location:`, `\r\n`, `email headers`, `rawHeaders`.

## Vulnerable Code Example:

```
// user-provided header value
String name = request.getParameter("name");
// directly set header with user input -> CR/LF can inject extra headers or response splitting
response.setHeader("X-User-Name", name);
```

## Secure Code Example:

```
String name = request.getParameter("name"); // read name
if (name != null) {
    name = name.replaceAll("[\\r\\n]", ""); // strip CR and LF characters from input
}
response.setHeader("X-User-Name", name); // set header with sanitized value
```

## 4.13. Host Header Injection

**Where to check**

- Code that uses `Host` header to build absolute URLs, generate password reset links, or to resolve tenant routing.

**What to check**

- Use of incoming Host header for URL generation without validation.
- Trusting host header for application logic (session cookie scope, routing).

**How to check**

- Search for code that reads request host header and uses it in link generation or security decisions.
- Ensure application sets canonical base URL from config, not from request.

**Vulnerable patterns**

- `var host = req.headers.host; url = "https://" + host + "/reset?token=" + token;`

**Secure patterns**

- Use known application config (canonical host) for generating absolute links; validate host header if used; set `server_name`/`trusted_proxies` and proper proxy headers.

**Search keywords**

- `req.headers.host`, `Host:`, `getHeader("Host")`, `request.getServerName()`.

### Vulnerable Code Example:

```
// read host header from request (attacker-controlled)
String host = request.getHeader("Host");
// build absolute URL using host header -> attacker can manipulate link
String url = "https://" + host + "/reset?token=" + token;
// send link potentially pointing to attacker-controlled domain
sendResetEmail(userEmail, url);
```

### Secure Code Example:

```
// get canonical host from trusted config
String canonicalHost = config.getCanonicalHost();
// build URL using config, not request header
String url = "https://" + canonicalHost + "/reset?token=" + token;
// send safe reset link
sendResetEmail(userEmail, url);
```

## 4.14. Null Byte Injection (via Host Header, LFI)

**Where to check**

- String-based path handling, legacy C/CGI bindings, libraries that treat null byte as terminator (esp. older platforms), header parsing.

**What to check**

- Use of raw bytes in path or header processing; absence of checks for `\0` in inputs.
- Languages/platforms where underlying C libraries handle null bytes specially.

**How to check**

- Inspect input handling for functions that convert to native bytes or pass to OS calls; ensure inputs are validated for control characters.
- Normalize inputs (strip null bytes) and validate after canonicalization.

**Vulnerable patterns**

- Passing user input directly to filesystem calls without removing control chars.

**Secure patterns**

- Reject or escape control characters; use safe language libraries that handle strings as first-class citizens.

**Search keywords**

- `\0`, `null byte`, `0x00`, `chr(0)`, `string.replace("\0", "")`.

## Vulnerable Code Example:

```
// passing raw bytes or relying on C-interop that treats nulls as terminators
// read filename
String filename = request.getParameter("file");
// naive concatenation could be problematic if null bytes mishandled by underlying libs
File f = new File("/data/" + filename);
```

## Secure Code Example:

```
// get filename
String filename = request.getParameter("file");
// detect null byte
if (filename != null && filename.indexOf('\0') >= 0) {
// reject if present
    throw new SecurityException("Null byte not allowed");
}
// canonical base path
Path base = Paths.get("/data").toRealPath();
// resolve and normalize
Path resolved = base.resolve(filename).normalize();
// prevent traversal
if (!resolved.startsWith(base)) throw new SecurityException("Invalid path");
```

## 4.15. HTTP Request Smuggling

**Where to check**

- Reverse proxy + backend setups, code that parses HTTP requests manually, fronting servers that strip/modify headers (Transfer-Encoding/Content-Length handling).

**What to check**

- Consistency between front-end proxy and backend server in parsing of Transfer-Encoding vs Content-Length.
- Any custom HTTP parsing code.

**How to check**

- Review infrastructure architecture and server versions/config (proxy + app server).
- Ensure proxies normalize or remove ambiguous headers and that application doesn't trust ambiguous request framing.
- Confirm use of vetted servers and latest patches.

**Vulnerable patterns**

- Code that relies on raw socket parsing or reuses untrusted raw request buffers.

**Secure patterns**

- Ensure proxies and servers are configured to reject ambiguous requests, disable or normalize conflicting headers, upgrade to patched versions.

**Search keywords**

- `Transfer-Encoding`, `Content-Length`, `proxy`, `chunked`, `Request smuggling`, `http parser`.

## Vulnerable Code Example:

```
// simplistic custom HTTP parsing trusting headers — vulnerable pattern
// read raw bytes from socket
InputStream in = socket.getInputStream();
// naive header read until CRLFCRLF
String headers = readUntilDoubleCRLF(in);
// simple header parsing that might not handle duplicates or ambiguous headers
Map<String,String> parsed = parseHeaders(headers);
// take Content-Length at face value
int contentLength = Integer.parseInt(parsed.get("Content-Length"));
// read body based on maybe inconsistent framing -> request smuggling risk
byte[] body = readBytes(in, contentLength);
```

## Secure Code Example:

```
// Use container / server for parsing; do not implement low-level parsing.
// If proxying, normalize and validate headers:
String te = request.getHeader("Transfer-Encoding"); // get Transfer-Encoding header
String cl = request.getHeader("Content-Length"); // get Content-Length header
if (te != null && cl != null) { // both present is ambiguous (chunked vs content-
length)
    response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Ambiguous framing"); //
reject ambiguous requests
    return; // stop processing
```

## 4.16. Prototype Pollution

**Where to check**

- JavaScript/Node code that merges user-controlled objects into prototypes, uses unsafe `extend` or deep-merge utilities with user input.

**What to check**

- Merging user input into objects, use of `_.merge`, `Object.assign` with user input, or setting keys like `__proto__`, `constructor.prototype`.

**How to check**

- Search for code that blindly merges request bodies into existing objects.
- Ensure libraries doing deep merges are patched and inputs are sanitized to prevent prototype keys.

**Vulnerable patterns**

- `Object.assign(config, req.body)` without sanitization.

**Secure patterns**

- Disallow keys like `__proto__`, `constructor` at boundary; use safe merge utilities; deep copy with whitelisting.

**Search keywords**

- `__proto__`, `constructor.prototype`, `Object.assign`, `_.merge`, `deepmerge`, `prototype pollution`.

### Vulnerable Code Example:

```
ScriptEngine engine = new ScriptEngineManager().getEngineByName("nashorn"); // create JS engine
String userScript = "var cfg = " + request.getParameter("cfg") + "; globalConfig = cfg;"; // embed user input into script
engine.eval(userScript); // evaluate user script -> may mutate global runtime or config -> unsafe
```

### Secure Code Example:

```
// Do not evaluate user-supplied scripts. If scripting is required, sandbox and validate inputs.
// get JSON config
String cfgJson = request.getParameter("cfg");
// Jackson object mapper
ObjectMapper om = new ObjectMapper();
// parse into typed object (no script execution)
Config cfg = om.readValue(cfgJson, Config.class);
// apply config in controlled manner (do not inject into global runtime)
applyConfig(cfg);
```

## 4.17. Unrestricted File Upload (MIME Sniffing / Content Sniffing)

**Where to check**

- File upload endpoints, image processors, document viewers, temp storage handling.

**What to check**

- Relying solely on filename or Content-Type header for validation.
- File storage paths, ability to upload executable scripts into webroot, transformation/previewing flows that may execute content.

**How to check**

- Verify server checks file signatures (magic bytes), enforces allowed types, filename normalization, size limits, scanning for malware.
- Ensure uploaded files are not served from a web-accessible directory with executable permissions; rename files and store outside webroot.

**Vulnerable patterns**

- Using `Content-Type` header as sole indicator or trusting file extension.

**Secure patterns**

- Verify magic bytes, enforce allow-list of MIME types, store outside webroot, randomize filenames, set restrictive file permissions, run virus scan, use content-disposition to force download, set `X-Content-Type-Options: nosniff`.

**Search keywords**

- `multipart/form-data`, `upload`, `Content-Type`, `file.save`, `filetype`, `mime`, `magic bytes`, `nosniff`.

**Vulnerable Code Example:**

```
// get uploaded file part
Part filePart = request.getPart("file");
// take original filename (untrusted)
String filename =
Paths.get(filePart.getSubmittedFileName()).getFileName().toString();
// write file into webroot with original name -> may upload executable or overwrite
files
filePart.write("/var/www/html/uploads/" + filename);
```

**Secure Code Example:**

```
// get uploaded file
Part filePart = request.getPart("file");
// open stream to read file
try (InputStream in = filePart.getInputStream()) {
// read header bytes to inspect magic bytes
    byte[] header = in.readNBytes(16);
// detect MIME based on magic bytes rather than header
    String mime = detectMime(header);
// enforce allow-list
    if (!allowedMime(mime)) throw new SecurityException("Disallowed file type");
// generate random storage name
    String storedName = UUID.randomUUID().toString() + ".dat";
// store outside webroot
    Path out = Paths.get("/var/data/uploads").resolve(storedName);

// write header
    Files.copy(new ByteArrayInputStream(header), out);
// write rest of file
    Files.copy(in, out, StandardCopyOption.APPEND);
    // store metadata in DB and serve via controller (not direct webroot), set X-
Content-Type-Options: nosniff
```

## 4.18. Deserialization (Insecure)

**Where to check**

- Deserialization calls: `pickle.loads`, `ObjectInputStream`, `Marshal.load`, `yaml.load` (unsafe), session deserialization code, message queue consumers.

**What to check**

- Use of unsafe deserializers with untrusted input; accepting binary blobs from network or cookies; presence of gadget classes that can be abused.

**How to check**

- Trace all deserialization entry points; ensure data is authenticated/verified (signatures) and from trusted sources.
- Prefer safe formats (JSON) or secure deserialization libraries or enable allow-lists of types.

**Vulnerable patterns**

- `pickle.loads(data)` on untrusted data; Java `ObjectInputStream` reading from network.

**Secure patterns**

- Avoid native object deserialization for untrusted data. Use JSON, protobuf with schema; if deserialization required, enforce strict type allow-lists and verify signatures/MAC.

**Search keywords**

- `deserialize`, `pickle.loads`, `ObjectInputStream`, `yaml.load`, `unmarshal`, `eval`, `load` + `socket/request`.

### Vulnerable Code Example:

```
// reading untrusted serialized object from network/cookie
// create ObjectInputStream on untrusted data
ObjectInputStream ois = new ObjectInputStream(someInputStream);
// deserializes arbitrary types -> may instantiate gadget classes -> RCE/logic
attacks
Object obj = ois.readObject();
```

### Secure Code Example:

```
// Use safer formats and filters
ObjectInputFilter filter =
ObjectInputFilter.Config.createFilter("com.example.MyAllowedClass;!*"); // allow-list
types
// create stream
ObjectInputStream ois = new ObjectInputStream(someInputStream);
// set filter to restrict types
ois.setObjectInputFilter(filter);
// safe-ish deserialization (only allowed classes)
Object obj = ois.readObject();
 // Better: prefer JSON with Jackson and disable polymorphic typing
MyDto dto = new ObjectMapper().readValue(jsonInput, MyDto.class); // use typed JSON
parsing instead of native deserialization
```

## 4.19. Regular Expression (ReDoS / Inefficient RegEx)

**Where to check**

- Input validation regexes, router path regexes, search features, log parsing, any user input validated by regex.

**What to check**

- Complex regexes that can take exponential time on specific inputs (catastrophic backtracking), unbounded quantifiers, unanchored patterns, heavy alternations.

**How to check**

- Review regexes for backtracking traps (nested quantifiers like `(a+)+`).
- Use static regex analyzers or fuzzers to detect worst-case inputs.
- Ensure timeouts or match limits enforced in libraries that support them.

**Vulnerable patterns**

- Unbounded `.*`, `(a+)+`, `(x|xx)+` with user-supplied strings.

**Secure patterns**

- Use atomic groups, possessive quantifiers where available, limit input length, prefer non-backtracking regex engines or parse with state machines.

**Search keywords**

- `regex`, `RegExp`, `match`, `replace` with regex, `.*`, `+`, `?`, `|`, `backtracking`, `ReDoS`, `catastrophic backtracking`.

### Vulnerable Code Example

```
// get query input
String input = request.getParameter("q");
// catastrophic backtracking example
Pattern p = Pattern.compile("^(a+)+$"); // regex with nested quantifiers ->
catastrophic backtracking
Matcher m = p.matcher(input); // create matcher
boolean matches = m.matches(); // running this on crafted input can cause CPU
exhaustion (ReDoS)
```

### Secure Code Example:

```
String input = request.getParameter("q"); // get input
if (input == null || input.length() > 500) { // bound input length to limit worst-
case time
    throw new IllegalArgumentException("Invalid input"); // reject oversized input
}
// use safer, anchored, non-backtracking patterns, or avoid regex for complex parsing
Pattern p = Pattern.compile("^[a]{1,500}$"); // avoid nested quantifiers by bounding
repetition
boolean matches = p.matcher(input).matches(); // safe match with bounded repetition
```